

Lab 8: Gaussian NBC

Step-by-Step Instructions:

```
In [1]: # Hello everyone, today, we'll walk through a Naive Bayes Classification example using synthetic sports data.
# We'll use pandas for data manipulation, matplotlib and seaborn for plotting,
# and sklearn for the Naive Bayes model and evaluation.

import pandas as pd # Importing pandas for data manipulation
import numpy as np # Importing numpy for numerical operations
from sklearn.model_selection import train_test_split # Importing train_test_split from sklearn for splitting the data
from sklearn.naive_bayes import GaussianNB # Importing GaussianNB from sklearn for Naive Bayes Classification
from sklearn.metrics import accuracy_score, precision_score, recall_score, confusion_matrix # Importing evaluation met
import matplotlib.pyplot as plt # Importing matplotlib for plotting
import seaborn as sns # Importing seaborn for advanced visualizations
import warnings # Importing warnings to manage warning messages

# We'll ignore any warnings to keep our output clean.
warnings.filterwarnings('ignore') # This will suppress any warning messages

# Let's start by loading our dataset into a pandas DataFrame.
data = pd.read_csv('synthetic_sports_data.csv') # Reading the CSV file into a DataFrame

# Displaying the first 5 rows of the dataset to get an idea of what it looks like.
print(data.head()) # Displaying the first 5 rows of the DataFrame

# Separating the independent variables (features) and dependent variable (target).
X = data[['Age', 'Training_Hours', 'Diet_Type', 'Previous_Wins']] # Selecting features
y = data['Win_Championship'] # Selecting the target variable

# Convert categorical variables into dummy/indicator variables.
X = pd.get_dummies(X)
```

	Age	Training_Hours	Diet_Type	Previous_Wins	Win_Championship
0	24	4	Balanced	11	Yes
1	37	8	Low Carb	3	No
2	32	7	Balanced	2	No
3	28	2	High Protein	19	No
4	25	1	Low Carb	11	Yes

```
In [2]: # Splitting the data into training and testing sets.
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42) # Splitting the data with 70%

# Creating a Naive Bayes model and fitting it to the training data.
model = GaussianNB() # Creating an instance of GaussianNB
```

```
model.fit(X_train, y_train) # Fitting the model to the training data

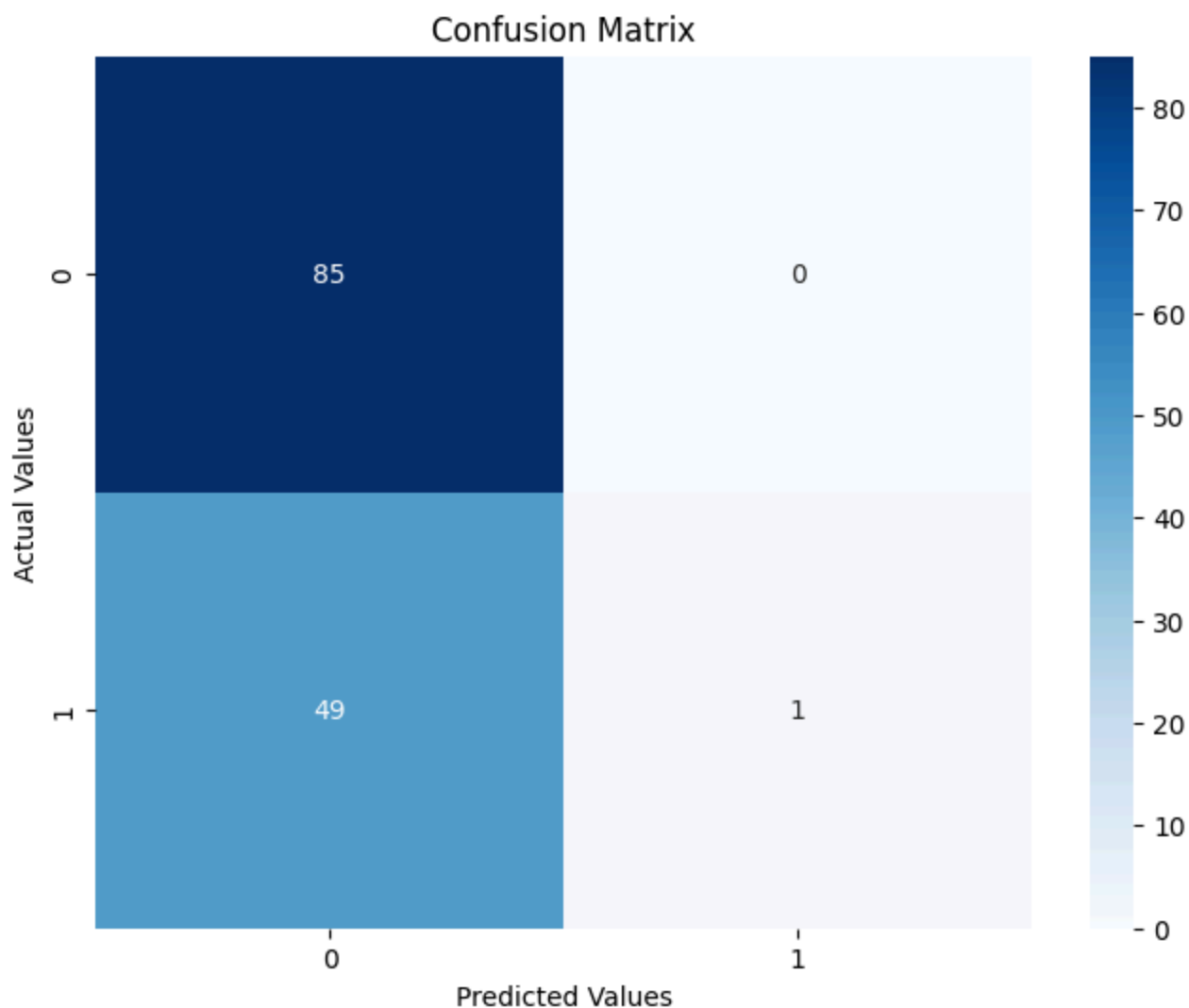
# Using the model to predict Championship Win on the test data.
y_pred = model.predict(X_test) # Making predictions on the test data
```

```
In [3]: # Calculating and printing the accuracy, precision, and recall of the model's predictions.
accuracy = accuracy_score(y_test, y_pred) # Calculating accuracy
precision = precision_score(y_test, y_pred, pos_label='Yes') # Calculating precision
recall = recall_score(y_test, y_pred, pos_label='Yes') # Calculating recall
print(f"Accuracy: {accuracy:.2f}") # Printing accuracy
print(f"Precision: {precision:.2f}") # Printing precision
print(f"Recall: {recall:.2f}") # Printing recall

# Calculating the confusion matrix.
conf_matrix = confusion_matrix(y_test, y_pred) # Calculating the confusion matrix

# Plotting the confusion matrix as a heatmap.
plt.figure(figsize=(8, 6)) # Creating a new figure with a specified size (8 inches by 6 inches)
sns.heatmap(conf_matrix, annot=True, fmt="d", cmap='Blues') # Creating a heatmap with the confusion matrix
plt.title('Confusion Matrix') # Adding a title to the heatmap
plt.ylabel('Actual Values') # Labeling the y-axis
plt.xlabel('Predicted Values') # Labeling the x-axis
plt.show() # Displaying the heatmap
```

Accuracy: 0.64
Precision: 1.00
Recall: 0.02



Let's take a look at the flow we followed for solving this problem

Import Libraries: We begin by importing necessary libraries including pandas for data manipulation, numpy for numerical operations, matplotlib and seaborn for plotting, and sklearn for machine learning tasks. We also import warnings to suppress any unwanted warnings.

Load and Inspect Data: We load the synthetic sports data into a DataFrame using pandas. We inspect the first few rows of the data to understand its structure and contents.

Prepare Data: We separate the independent variables (features) and the dependent variable (target) from the dataset. We convert categorical variables (Diet_Type) into dummy/indicator variables for the model to process.

Split Data: We split the data into training and testing sets using `train_test_split`. Here, 70% of the data is used for training the model, and 30% is used for testing.

Train Model: We create an instance of the `GaussianNB` model for Naive Bayes classification. We fit the model to the training data using the `fit` method.

Make Predictions: We use the trained model to make predictions on the test data using the `predict` method.

Evaluate Model: We evaluate the model's performance by calculating accuracy, precision, and recall scores. We also calculate the confusion matrix to summarize prediction results.

Visualize Results: We visualize the confusion matrix using a heatmap for better understanding and interpretation. The heatmap displays the actual vs. predicted values, helping us see where the model is performing well and where it may be making mistakes.

```
In [25]: from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
In [10]: df = pd.read_csv("train.csv")
df.head()
```

```
Out[10]:
```

	battery_power	blue	clock_speed	dual_sim	fc	four_g	int_memory	m_dep	mobile_wt	n_cores	...	px_height	px_width	ram
0	842	0	2.2	0	1	0	7	0.6	188	2	...	20	756	2549
1	1021	1	0.5	1	0	1	53	0.7	136	3	...	905	1988	2631
2	563	1	0.5	1	2	1	41	0.9	145	5	...	1263	1716	2603
3	615	1	2.5	0	0	0	10	0.8	131	6	...	1216	1786	2769
4	1821	1	1.2	0	13	1	44	0.6	141	2	...	1208	1212	1411

5 rows × 21 columns

```
In [11]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 21 columns):
#   Column          Non-Null Count  Dtype
---  -
0   battery_power    2000 non-null   int64
1   blue             2000 non-null   int64
2   clock_speed      2000 non-null   float64
3   dual_sim         2000 non-null   int64
4   fc               2000 non-null   int64
5   four_g           2000 non-null   int64
6   int_memory       2000 non-null   int64
7   m_dep            2000 non-null   float64
8   mobile_wt        2000 non-null   int64
9   n_cores          2000 non-null   int64
10  pc               2000 non-null   int64
11  px_height        2000 non-null   int64
12  px_width         2000 non-null   int64
13  ram              2000 non-null   int64
14  sc_h             2000 non-null   int64
15  sc_w             2000 non-null   int64
16  talk_time        2000 non-null   int64
17  three_g          2000 non-null   int64
18  touch_screen     2000 non-null   int64
19  wifi             2000 non-null   int64
20  price_range      2000 non-null   int64
dtypes: float64(2), int64(19)
memory usage: 328.2 KB
```

```
In [12]: df['price_range'].value_counts()
```

```
Out[12]: price_range
1      500
2      500
3      500
0      500
Name: count, dtype: int64
```

```
In [14]: X = df.drop('price_range', axis=1)
y = df['price_range']
```

```
In [17]: scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
In [18]: X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)
```

```
In [22]: model = LogisticRegression(multi_class='multinomial', solver='lbfgs', max_iter=500)
model.fit(X_train, y_train)
```

```
Out[22]: LogisticRegression
LogisticRegression(max_iter=500, multi_class='multinomial')
```

```
In [23]: y_pred = model.predict(X_test)
```

```
In [29]: print("Accuracy:", accuracy_score(y_test, y_pred))
print("-"*55)
print("Classification Report:\n", classification_report(y_test, y_pred))
```

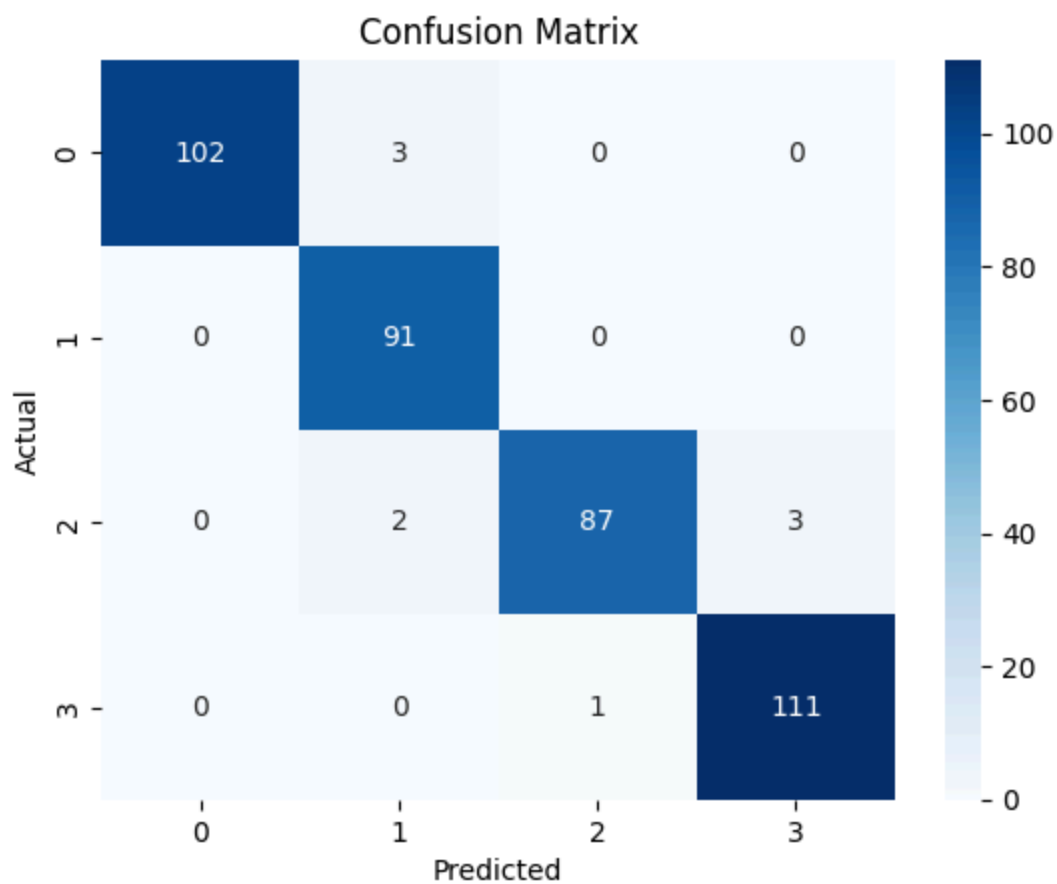
Accuracy: 0.9775

```
-----
Classification Report:
              precision    recall  f1-score   support

     0           1.00        0.97        0.99         105
     1           0.95        1.00        0.97          91
     2           0.99        0.95        0.97          92
     3           0.97        0.99        0.98         112

 accuracy                   0.98         400
 macro avg                0.98         0.98         0.98         400
 weighted avg             0.98         0.98         0.98         400
```

```
In [32]: cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, cmap='Blues', fmt='g')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```



Interpretation

- Trained a logistic regression model to classify mobile phones into four price ranges (0–3) based on features like RAM, battery, and display size.
- Achieved 97.75% accuracy with consistently high precision, recall, and F1-scores across all classes.
- Minor misclassifications occurred mostly between adjacent price categories (e.g., class 2 vs 3).
- Learned how to handle multi-class classification, apply feature scaling, and evaluate model performance using a confusion matrix and classification report.

In []: